UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

GUÍA DE LABORATORIO N.º 1

Fundamentos y Entornos de Desarrollo en Python

I. Propósito de la sesión

Al finalizar el laboratorio, el estudiante será capaz de configurar un entorno de desarrollo Python profesional y reproducible: aislar dependencias con un entorno virtual, versionar su código con Git siguiendo convenciones de la industria, y organizar un proyecto con una estructura de carpetas y documentación estándar.

II. Requisitos previos

- Python 3.10 o superior instalado (verificar con python --version).
- Git instalado (verificar con git --version).
- Editor de código Visual Studio Code, con la extensión oficial de Python.
- Cuenta activa en GitHub (github.com).
- Conexión a internet para instalar paquetes y sincronizar el repositorio remoto.

III. Estructura de la sesión (4 horas)

Parte	Tema	Nivel	Duración
1	Entornos virtuales y gestión de dependencias	Básico	60 min
2	Control de versiones con Git aplicado a Python	Intermedio	90 min
3	Estructura de proyecto, PEP 8 y consolidación	Avanzado	90 min
Total			**240 min**

IV. Desarrollo paso a paso

PARTE 1 — ENTORNOS VIRTUALES Y GESTIÓN DE DEPENDENCIAS

Paso 1.1 — Verificación de requisitos

Abra una terminal (símbolo del sistema, PowerShell o terminal de VS Code) y ejecute:

```
python --version
pip --version
git --version
```

Si el comando python no responde en Windows, intente con py --version.

Paso 1.2 — Creación de la carpeta de proyecto y del entorno virtual

```
mkdir tecemer-lab1
cd tecemer-lab1
python -m venv .venv
```

Active el entorno según su sistema operativo:

```
# Windows (PowerShell)
.venv\Scripts\activate

# Linux / macOS
source .venv/bin/activate
```

Verificación: *Si la terminal muestra el prefijo (.venv) antes de la ruta, el entorno está activo correctamente.*

Paso 1.3 — Instalación de paquetes y registro de dependencias

Con el entorno activo, instale la librería requests:

```
pip install requests
pip list
```

Genere un registro de dependencias (forma clásica, útil como respaldo):

```
pip freeze > requirements.txt
```

Paso 1.4 — Mini-reto: Primer script funcional

Cree un archivo main.py con el siguiente contenido, que consume una API pública para comprobar que el entorno funciona correctamente:

```
import requests

respuesta = requests.get("https://official-joke-api.appspot.com/random_joke")
datos = respuesta.json()
print(datos["setup"])
print(datos["punchline"])
```

```
python main.py
```

Verificación: *Checkpoint de Parte 1: el script debe imprimir un chiste en dos líneas sin errores. Levante la mano si obtiene un traceback antes de continuar.*

PARTE 2 — CONTROL DE VERSIONES CON GIT APLICADO A PYTHON

Paso 2.1 — Configuración inicial de Git

```
git config --global user.name "Su Nombre"
git config --global user.email "su_correo@uncp.edu.pe"
git init
```

Paso 2.2 — Creación de .gitignore y primer commit

Cree un archivo .gitignore en la raíz del proyecto con el siguiente contenido:

```
.venv/
__pycache__/
*.pyc
.env
*.egg-info/
dist/
build/
.pytest_cache/
```

```
git add .gitignore requirements.txt main.py
git commit -m "chore: configura entorno virtual y primer script funcional"
```

Paso 2.3 — Trabajo con ramas

Cree una rama de feature para agregar manejo de errores al script:

```
git checkout -b feature/manejo-errores
```

Modifique main.py agregando control de excepciones:

```
import requests

try:
    respuesta = requests.get(
        "https://official-joke-api.appspot.com/random_joke", timeout=5
    )
    respuesta.raise_for_status()
    datos = respuesta.json()
    print(datos["setup"])
    print(datos["punchline"])
except requests.RequestException as error:
    print(f"No se pudo obtener el chiste: {error}")
```

```
git add main.py
git commit -m "feat: agrega manejo de errores en la consulta a la API"
```

Paso 2.4 — Repositorio remoto y Pull Request en GitHub

1. En GitHub, cree un repositorio vacío llamado tecemer-lab1 (sin README, sin .gitignore).
2. Conecte su repositorio local con el remoto y suba la rama main.
3. Suba también la rama feature/manejo-errores.
4. En la interfaz web de GitHub, abra un Pull Request de feature/manejo-errores hacia main.
5. Revise los cambios mostrados (diff) y fusione (Merge) el Pull Request.
6. Actualice su copia local de main.

```
git remote add origin https://github.com/SU-USUARIO/tecemer-lab1.git
git push -u origin main
git push -u origin feature/manejo-errores
# ... abrir y fusionar el Pull Request en github.com ...
git checkout main
git pull origin main
```

Paso 2.5 — Checkpoint de Parte 2

Verifique en GitHub que su repositorio muestra al menos 2 commits en el historial de main, con mensajes en formato Conventional Commits (chore:, feat:, etc.), y que el Pull Request aparece como fusionado (Merged).

PARTE 3 — ESTRUCTURA DE PROYECTO, PEP 8 Y CONSOLIDACIÓN

Paso 3.1 — Migración a src layout

Reorganice el proyecto separando el código importable en una subcarpeta:

```
mkdir -p src/tecemer_lab1
move main.py src/tecemer_lab1/app.py    # Windows: usar 'move'; Linux/macOS: 'mv'
type nul > src/tecemer_lab1/__init__.py  # Windows
touch src/tecemer_lab1/__init__.py       # Linux/macOS
```

La estructura resultante debe verse así:

```
tecemer-lab1/
├── src/
│   └── tecemer_lab1/
│       ├── __init__.py
│       └── app.py
├── .venv/          (no versionado)
├── .gitignore
├── pyproject.toml
├── requirements.txt
└── README.md
```

Paso 3.2 — pyproject.toml e instalación editable

Cree un archivo pyproject.toml en la raíz con el siguiente contenido:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "tecemer-lab1"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "requests>=2.28",
]
```

```
pip install -e .
```

Verificación: *Con la instalación editable, puede importar el paquete como 'from tecemer_lab1 import app' desde cualquier script dentro del entorno, sin reinstalarlo tras cada cambio.*

Paso 3.3 — Estilo de código con PEP 8

```
pip install black pycodestyle
```

Primero identifique observaciones de estilo:

```
pycodestyle src/
```

Luego formatee automáticamente con Black y revise los cambios:

```
black src/
git diff
```

```
git add pyproject.toml src/
git commit -m "style: aplica formateo PEP 8 con Black y agrega pyproject.toml"
```

Paso 3.4 — Documentación con README.md

Redacte un README.md con, al menos, las siguientes secciones:

- Título y descripción breve del proyecto.
- Instrucciones de instalación (creación del entorno virtual e instalación editable).
- Ejemplo de uso (cómo ejecutar app.py).
- Estructura del repositorio.
- Autor y curso (Tecnologías Emergentes — ISO46B).

```
git add README.md
git commit -m "docs: agrega README con instrucciones de instalación y uso"
```

Paso 3.5 — Consolidación final y entrega

```
git push origin main
```

Copie el enlace de su repositorio de GitHub y súbalo al aula virtual como evidencia de entrega.

V. Entregable de la sesión

Repositorio en GitHub con: estructura en src layout, entorno virtual documentado (no subido al repositorio), pyproject.toml funcional con instalación editable, código formateado según PEP 8, README.md completo, y un historial de al menos 5 commits en formato Conventional Commits, incluyendo al menos una rama de feature fusionada mediante Pull Request.